SPRING QC — ENGINEERING GUIDE

Deploying and Packaging Spring QC

A plain-language guide to running the system on-site: which deployment format to use, and what it would take to turn it into a Windows .exe.

Prepared	23 September 2026
Audience	Anyone deciding how Spring QC gets installed and run on-site – no Docker or Python background assumed
Covers	Deployment format comparison, and native .exe packaging options

00 WHAT THIS SYSTEM IS, IN ONE PICTURE

What this system is, in one picture

Spring QC watches springs move past a camera, decides whether each one is inside a marked zone (the “ROI”), and asks a human operator to confirm whether it’s good or defective. One computer – the **edge box** – does the actual video analysis (the AI model). Everyone else just uses a web browser pointed at it:

Edge box	One PC near the line, with a graphics card (GPU). Runs the database and the AI model. This is the only machine doing heavy work.
Admin	Any laptop/PC on the same local network. Opens a browser to the edge box to configure cameras, operators, and settings.
Operator panel	A touchscreen at the machine. Opens a browser to the edge box, full-screen, so the operator can mark each spring good or defective.

Everything is on the **same local network** (a LAN) – no internet connection needed, and none of the video ever has to travel further than the building it’s in. This guide answers two practical questions about that edge box: *what software format should it run in*, and *could it be a simple double-click .exe instead*.

01 PART 1 – WHICH DEPLOYMENT FORMAT TO USE

Part 1 – Which deployment format to use

There are three realistic ways to get Spring QC running on the edge box. In plain terms:

- **Docker Compose** – the app and its database run inside two lightweight, sealed “boxes” (containers) that ship with everything they need already inside them. This is what has actually been built and tested for this project.
- **Bare-metal Python** – the app is installed directly onto Windows, the normal way you’d install any program, sharing the computer’s existing Python and libraries.
- **A native .exe installer** – a single click-through Windows installer, like installing any commercial desktop software, with no Docker or Python visible to the person installing it.

1.1 Scorecard

A side-by-side comparison across everything that matters for a machine that runs unattended on a factory floor, with green = good, amber = workable, red = weak spot.

	Docker Compose	Bare-metal Python	Native .exe
Ready today	🟢 Built & verified (GPU and CPU)	🟠 Scripts exist, not the primary path	🔴 Not started – needs a full build
Skill needed on-site	🟠 Basic Docker commands	🔴 Python/venv troubleshooting	🟢 Click through an installer
Keeps dependencies separate	🟢 Fully contained	🔴 Shares the host’s Python	🔴 Installs into the host
Restarts itself after a crash/reboot	🟢 Built in	🔴 Needs a hand-built wrapper	🟠 Needs a Windows Service add-on
Updating the app	🟢 Rebuild image, a few minutes	🟠 Re-run setup on the live machine	🔴 New multi-GB installer each time
Undoing a bad update	🟢 Swap back to the old version	🔴 No safety net	🔴 Reinstall the previous build
Extra software needed	🟠 Docker Desktop (~1 GB)	🟢 None beyond Python + database	🟠 None – the installer carries it
Same setup at every site	🟢 Identical every time	🔴 Drifts with each manual setup	🟢 Identical every time

1.2 Each format in detail: what you gain, what it costs

A Docker Compose

BEST FIT

What's running today – the app and database sealed into containers.

WHAT YOU GAIN

- Already built, tested, and proven end to end on this exact app – both the GPU and CPU versions work.
- The AI libraries (PyTorch, CUDA, OpenCV) are sealed away from the rest of the computer, so a Windows update or another program being installed can't quietly break the model server.
- If it crashes or the box loses power, it comes back on its own – no one has to notice and restart it by hand.
- Updating the model or a setting is a quick rebuild, and if something goes wrong you can switch straight back to the version that was working.
- The exact same setup runs identically at a second site – no surprises from one installation to the next.

WHAT IT COSTS YOU

- Docker itself has to be installed on the edge box first – about 1 GB, with its own update cycle to keep an eye on.
- On Windows it relies on an extra layer underneath (WSL2), which can occasionally need a restart after a big Windows update before everything comes back up.
- Getting the graphics card working inside a container is one extra setup step, done once per box.
- Troubleshooting means being comfortable with a handful of Docker commands, not just double-clicking an icon.

B Bare-metal Python

WEAKEST FIT

Installing Python and the database directly onto Windows, the traditional way.

WHAT YOU GAIN

- Nothing extra running underneath – one less layer between the app and the graphics card.
- Slightly less memory and disk used at idle than running Docker on top of everything.
- The setup scripts for this already exist from early development, so it isn't starting completely from scratch.

WHAT IT COSTS YOU

- No separation at all: anything else ever installed on that computer, or any Windows update, can break the exact software versions this app depends on.
- Nothing restarts it automatically after a crash – that has to be built by hand, and even then it won't be as polished as a proper installer.
- Every edge box is set up manually by whoever is on-site that day – small differences between sites become the norm, not the exception.
- Updating means changing files on a live, working machine with no safety net if it goes wrong partway through.

C Native .exe installer

FALLBACK ONLY

A future option – a click-through Windows installer with no Docker or Python required.

WHAT YOU GAIN

- The easiest possible experience for whoever installs it on-site – no technical background needed, just run the installer.
- No dependency on Docker at all – the one genuine reason to build this instead of using Docker Compose.
- Feels like “real” installed software: a Start Menu entry, a version number, a proper uninstaller.

WHAT IT COSTS YOU

- Doesn’t exist yet – this is the most work to build, and the most ongoing work to keep updated alongside everything else.
- Same lack of separation as bare-metal – it installs straight onto the computer with no protective boundary.
- A graphics-card version of the installer is multiple gigabytes – slow to produce and to redistribute every time something changes.
- No quick way to undo an update – reverting means reinstalling the previous version by hand.

Recommendation for Part 1

Keep using Docker Compose. It is the only one of the three formats that has actually been built and proven against this app – the GPU inference, the database, and both dashboards all work today. It is also the only format that protects the finicky AI libraries from anything else that happens to that computer over its lifetime.

Save the native .exe installer for a specific site that outright forbids Docker – treat it as a deliberate exception, not the default. Bare-metal Python is the weakest choice for something meant to run unattended on a factory floor: no separation, no automatic recovery, and no safety net if an update goes wrong.

02 PART 2 – COULD THIS BE A SIMPLE .EXE?

Part 2 – Could this be a simple .exe?

A separate, related question: could the system be turned into a normal double-click **.exe** file, the way most Windows software works? The honest answer is that “make an .exe” means two quite different things here, because only one part of the system actually does heavy work.

2.1 Two different things could become an .exe

Path A	**The edge box itself, as an .exe** Replaces Docker on the one computer doing the AI work. Has to carry Python, the AI libraries, the graphics-card software, and the database. This is the heavy, expensive option.
Path B	**A simple window app, as an .exe** Replaces “open Chrome in full-screen mode” on the admin PC or the operator touchscreen. It’s just a window showing the edge box’s web page – no AI libraries at all. This is the light, cheap option.

2.2 Path A: packaging the AI server – five real decisions

If this route is taken, five separate choices each carry a genuine trade-off:

Decision	The choice	The trade-off
Packaging tool	PyInstaller vs. Nuitka	PyInstaller is the standard for this; Nuitka doesn’t actually shrink the AI libraries, which are the real size problem either way.
Single file or a folder	“Onedir” (recommended) vs. “onefile”	A single giant .exe re-unpacks itself to a temp folder *every time* it starts, and looks more suspicious to antivirus software.
Graphics card support	CPU-only vs. GPU build	CPU-only is small but brings back the video-lag problem already solved by using a GPU; a GPU build is several gigabytes.
The database	Bundle Postgres, or fall back to a simpler built-in database	Bundling keeps today’s design but means managing a second background program; a simpler built-in database is easier to package but is a step back from the current design.
Starting automatically	Plain .exe vs. a Windows Service	A plain .exe only runs while someone is logged in; matching Docker’s “always comes back on” behaviour needs it registered as a Windows Service.

The installer itself – the Start Menu icon, the uninstaller, optionally setting up the database – is built with **Inno Setup**, a free, widely used tool for this exact job on Windows.

2.3 Path B: packaging the window app – much simpler

Tool	What it is	Installer size	
pywebview	Uses the web-browser engine already built into Windows 10/11 (“WebView2”) inside a plain window.	~5 MB	RECOMMENDED
Tauri	The same idea, built with a different toolset.	~10 MB	VIABLE
Electron	Ships its own full copy of the Chrome browser inside the app.	150 MB+	SKIP

This gives the admin and each operator panel a proper app icon and a locked-down window – no address bar, no way to accidentally browse away from the app – which is a genuinely nicer replacement for “Chrome in kiosk mode.” It does not touch the AI server at all, so it carries almost no risk.

Recommendation for Part 2

Don’t rebuild the AI server as an .exe (Path A) unless Docker is specifically the thing blocking a site – the current Docker setup already solves the problems that path would re-solve, for a fraction of the build and upkeep effort.

If a nicer app icon on the admin PC or operator panel is wanted, **Path B alone** is worth doing – it’s a few days of low-risk work and doesn’t touch the working AI pipeline at all.

03 PLAIN-LANGUAGE GLOSSARY

Plain-language glossary

Edge box	The one computer, physically near the production line, that runs the AI model and the database. Everything else just talks to it over the network.
LAN	Local Area Network – the site's own network (switches, Wi-Fi, cabling). No internet connection is required for this system to work.
GPU / graphics card	The hardware that makes the AI model run fast enough to keep up with a live camera feed. Without one, video from a live camera can noticeably lag.
Docker / container	A way of packaging software so it carries everything it needs with it, sealed off from whatever else is installed on the computer – like a shipping container that works the same no matter which ship it's loaded onto.
Kiosk mode	A browser setting that shows a web page full-screen with no address bar, tabs, or way to navigate elsewhere – used to lock the operator touchscreen to just the one screen it needs to show.
Windows Service	A program registered to run in the background automatically, even before anyone logs in, and to restart itself if it stops – the standard way to make unattended software reliable on Windows.
.exe / installer	A standard double-click Windows program or setup file, the same format as any commercial desktop software.
Rollback	Undoing an update by switching back to the previous working version – easy with Docker (swap a version tag), hard with a manual install (reinstall by hand).

Spring QC – deployment engineering | companion to the two on-screen briefs ("Spring QC Packaging" and "Spring QC Deployment Formats")